Terraform Provisioners are used to performing certain custom actions and tasks either on the local machine or on the remote machine.

The custom actions can vary in nature and it can be -
1.Running custom shell script on the local machine
2.Running custom shell script on the remote machine
3.Copy file to the remote machine

Also there are two types of provisioners -
1.Generic Provisioners (file, local-exec, and remote-exec)
2.Vendor Provisioners (chef, habitat, puppet, salt-masterless)

Generic Provisioners - Generally vendor independent and can be used with any cloud vendor(GCP, AWS, AZURE)
Vendor Provisioners - It can only be used only with its vendor. For example, chef provisioner can only be used with chef for automating and provisioning the server configuration.

the essential Terraform Cheatsheet

by justin o'connor

general commands

get the terraform version
```
terraform version
```

download and update root modules
```
terraform get -update=true
```

open up a terraform interactive terminal
```
terraform console
```

create a dot diagram of terraform dependencies
```
terraform graph | dot -Tpng > graph.png
```

format terraform code to HCL standards
```
terraform fmt
```

validate terraform code syntax
```
terraform validate
```

enable tab auto-completion in the terminal
```
terraform -install-autocomplete
```

show infromation about provider requirements
```
terraform providers
```

login and logout of terraform cloud
`terraform login` **and** `terraform logout`

workspaces

list the available workspaces
```
terraform workspace list
```

create a new workspace
```
terraform workspace new development
```

select an existing workspace
```
terraform workspace select default
```

initilize terraform

initialize terraform in the current working directory
```
terraform init
```

skip plugin installation
```
terraform init -get-plugins=false
```

force plugin installation from a directory
```
terraform init -plugin-dir=PATH
```

upgrade modules and plugins at initilization
```
terraform init -upgrade
```

update backend configuration
```
terraform init -migrate-state -force-copy
```

skip backend configuration
```
terraform init -backend=false
```

use a local backend configuration
```
terraform init -backend-config=FILE
```

change state lock timeout (default is zero seconds)
```
terraform init -lock-timeout=120s
```

plan terraform

produce a plan with diff between code and state
```
terraform plan
```

output a plan file for reference during apply
```
terraform plan -out current.tfplan
```

output a plan to show effect of terraform destroy
```
terraform plan -destroy
```

target a specific resource for deployment
```
terraform plan -target=ADDRESS
```

note that the -target option is also available for the terraform apply and terraform destroy commands.

outputs

list available outputs
```
terraform output
```

output a specific value
```
terraform output NAME
```

apply terraform

apply the current state of terraform code
```
terraform apply
```

specify a previously generated plan to apply
```
terraform apply current.tfplan
```

enable auto-approval or automation
```
terraform apply -auto-approve
```

destroy terraform

destroy resources managed by terraform state
```
terraform destroy
```

enable auto-approval or automation
```
terraform destroy -auto-approve
```

manage terraform state

list all resources in terraform state
```
terraform state list
```

show details about a specific resource
```
terraform state show ADDRESS
```

track an existing resource in state under new name
```
terraform state mv SOURCE DESTINATION
```

import a manually created resource into state
```
terraform state import ADDRESS ID
```

pull state and save to a local file
```
terraform state pull > terraform.tfstate
```

push state to a remote location
```
terraform state push PATH
```

replace a resource provider
```
terraform state replace-provider A B
```

taint a resource to force redeployment on apply
```
terraform taint ADDRESS
```

untaint a prevoiusly tainted resource
```
terraform untaint ADDRESS
```

1. File privileges
2. Local privileges
3. Zone-wide privileges

1. file provisioner

As the name suggests *file provisioner* can be used for transferring and copying the files from one machine to another machine.

Not only file but it can also be used for transferring/uploading the directories.

So when we talk about copying files or directories from one machine to another machine then it has to be secured and *file provisioner* supports for `ssh` and `winrm` type of connections which can help you to achieve secure file transfer between the source machine and destination machine.

Let us take an example to understand how to implement terraform file provisioner. The following code snippet shows -

1. How to write your file provisioner
2. How to specify `source and` destination` for copying/transferring the file.

```
provisioner "file" {
    source      = "/home/rahul/Jhooq/keys/aws/test-file.txt"
    destination = "/home/ubuntu/test-file.txt"
}
```

Here is the complete terraform script which demonstrates on how to use *terraform file provisioner*

BASH

```
provider "aws" {
   region     = "eu-central-1"
   access_key = "AKIATQ37NXBxxxxxxxxx"
   secret_key = "JzZKiCia2vjbq4zGGGewdbOhnacmxxxxxxxxxxx"

}

resource "aws_instance" "ec2_example" {

    ami = "ami-0767046d1677be5a0"
    instance_type = "t2.micro"
    key_name= "aws_key"
    vpc_security_group_ids = [aws_security_group.main.id]

  provisioner "file" {
    source      = "/home/rahul/Jhooq/keys/aws/test-file.txt"
    destination = "/home/ubuntu/test-file.txt"
  }
  connection {
      type        = "ssh"
      host        = self.public_ip
      user        = "ubuntu"
      private_key = file("/home/rahul/Jhooq/keys/aws/aws_key")
      timeout     = "4m"
   }
}

resource "aws_security_group" "main" {
  egress = [
    {
      cidr_blocks      = [ "0.0.0.0/0", ]
      description      = ""
      from_port        = 0
      ipv6_cidr_blocks = []
      prefix_list_ids  = []
      protocol         = "-1"
      security_groups  = []
      self             = false
      to_port          = 0
    }
  ]
 ingress                = [
   {
     cidr_blocks      = [ "0.0.0.0/0", ]
     description      = ""
     from_port        = 22
     ipv6_cidr_blocks = []
     prefix_list_ids  = []
     protocol         = "tcp"
     security_groups  = []
...
```

Supporting arguments for file provisioners

1. source - The source argument is used to specify the location from where you want to pick the file. The source location can be relative to your project structure.

Here are some examples where I have used relative path for the source arguments -

BASH

```
provisioner "file" {
    source      = "../../../Jhooq/keys/aws/test-file.txt"
    destination = "/home/ubuntu/test-file.txt"
}
```

2. content - The content argument is useful when you do not want to copy or transfer the file instead you only want to copy the `content` or `string`.

Here is an example of a content resource argument -

```BASH
provisioner "file" {
    content     = "I want to copy this string to the destination file server.txt"
    destination = "/home/ubuntu/server.txt"
}
```

The above provisioner script will copy the string `I want to copy this string to the destination file server.txt` to the destination file `/home/ubuntu/server.txt`

3. destination - As the name suggest you need to input the final destination path where you want your file to be.

2. local-exec provisioner

The next provisioner we are gonna talk about is *local-exec provisioner*. Basically, this provisioner is used when you want to perform some tasks onto your local machine where you have installed the terraform.

So `local-exec` provisioner is never used to perform any kind task on the remote machine. It will always be used to perform local operations onto your local machine.

Example - Consider the following example where we are trying to create a file `hello-jhooq.txt` on the local machine

```bash
provisioner "local-exec" {
    command = "touch hello-jhooq.txt"
}
```

In the command section, we can write a bash script. In the above example, I am trying to create a `hello-jhooq.txt` file on the local machine.

Here is the complete terraform script for the above example -

```BASH
provider "aws" {
   region     = "eu-central-1"
   access_key = "AKIATQ37NXBxxxxxxxxx"
   secret_key = "JzZKiCia2vjbq4zGGGewdbOhnacmxxxxxxxxxxx"

}

resource "aws_instance" "ec2_example" {

    ami = "ami-0767046d1677be5a0"
    instance_type = "t2.micro"
    tags = {
        Name = "Terraform EC2"
    }

  provisioner "local-exec" {
    command = "touch hello-jhooq.txt"
  }
}
```

Supporting arguments for local provisioners

1. command - Here are the key facts about the ***command*** arguments

1. This is a mandatory argument which you always need to pass along whenever you are implementing the `local-exec` provisioners.

2. Always consider command as shell script executioner because whatever you pass in the command will be executed as a bash shell script.

3.You can write even mention the relative path of your shell script location and pass it the command.

```BASH
provisioner "local-exec" {
    command = "touch hello-jhooq.txt"
}
```

2. working_dir - Here are the key facts about the `working_dir` arguments

1. It is an optional argument which and you do not necessarily need to pass along with the command argument.

2. This is a supporting argument for the `command` because once you specify the `working_dir` you are explicitly telling terraform to execute the command at that particular location.

3. You can mention the relative path of the `working_dir`.

3. interpreter - With the help of `interpreter` you can explicitly specify in which environment(bash, PowerShell, perl etc.) you are going to execute the command.

1. It is an optional argument.
2. If you do not specify the `interpreter` argument the default will be taken into consideration based on the operating system.

Example 1: - Here I am trying to specify the interpreter as perl, so anything which I mention inside the command argument will be executed as perl command

```bash
resource "null_resource" "example1" {
  provisioner "local-exec" {
    command = "open WFH, '>hello-world.txt' and print WFH scalar localtime"
    interpreter = ["perl", "-e"]
  }
}
```

Example 2: - In this example I will be using the `PowerShell` interpreter to write a string to a file

```bash
resource "null_resource" "example2" {
  provisioner "local-exec" {
    command = "This will be written to the text file> completed.txt"
    interpreter = ["PowerShell", "-Command"]
  }
}
```

4. environment - This is again an optional parameter that can be passed alongside the `command` argument.

1. With the help of `environment` you can define or set environment variables that can be accessible later or inside your terraform execution.

2. `environment` arguments are generally the key-value pair and you can define as many variables as you can.

Here is an example of the `environment` arguments

```bash

provisioner "local-exec" {
    command = "echo $VAR1 $VAR2 $VAR3 >> my_vars.txt"

    environment = {
      VAR1 = "my-value-1"
      VAR2 = "my-value-2"
      VAR3 = "my-value-3"
    }
  }
```

3. remote-exec provisioner

As the name suggests `remote-exec` it is always going to work on the remote machine. With the help of the `remote-exec` you can specify the commands of shell scripts that want to execute on the remote machine.

As we discussed `ssh` and `winrm` for secure data transfer in local-exec, here also all the communication and file transfer is done securely.

Let us take an example of how to implement the `remote-exec` provisioner -

```BASH
provisioner "remote-exec" {
    inline = [
      "touch hello.txt",
      "echo helloworld remote provisioner >> hello.txt",
    ]
}
```

In the above example -

1. First we are going to create a file named `hello.txt`
2. We are going to write the message `helloworld remote provisioner` inside the `hello.txt` file.
3. Everything will happen on the remote machine

Supporting arguments for remote provisioners

1. inline - With the help of an inline argument you can specify the multiple commands which you want to execute in an ordered fashion.

Here is an example in which I have added two separate commands -

```BASH
provisioner "remote-exec" {
    inline = [
      "touch hello.txt",
      "echo helloworld remote provisioner >> hello.txt",
    ]
}
```

2. script - It can be used to copy the script from local machine to remote machine and it always contains a relative path.

In the script, you can not specify multiple scripts. You can only mention one script which needs to be copied to the remote machine.

3. scripts - Here you can specify the multiple local scripts which want to copy or transfer to the remote machine and execute over there.

Always remember the order of the file will not change and it going to execute in the same order way you have mentioned.

4. chef provisioner

Now are talking about vendor-specific provisioners and the first one in our list is ***chef***.

chef is an infrastructure automating open-source tool for managing and automating the servers remotely. Terraform has really good integration with ***chef***.

If you are using ***chef*** for managing your infrastructure then you can take the advantage of the ***chef provisioner*** but always use provisioners in the moderation and when it is necessary.

As with ***chef*** we are handling the remote servers so every communication is happening over either `ssh` or `winrm`.

Here is an example code snippet of the ***chef*** provisioner -

```BASH
resource "aws_instance" "chef_provisioner_example" {

  provisioner "chef" {
    attributes_json = <<EOF
      {
        "key": "value",
        "app": {
          "cluster1": {
            "nodes": [
              "webapp1",
              "webapp2"
            ]
          }
        }
      }
    EOF

    environment     = "_default"
    client_options  = ["chef_license 'accept'"]
    run_list        = ["cookbook::recipe"]
    node_name       = "webapp1"
    secret_key      = "${file("../encrypted_data_bag_secret")}"
    server_url      = "https://chef.company.com/organizations/org1"
    recreate_client = true
    user_name       = "bork"
    user_key        = "${file("../bork.pem")}"
    version         = "15.10.13"
    # If you have a self signed cert on your chef server change this to :verify_none
    ssl_verify_mode = ":verify_peer"
  }
}
```

5. habitat provisioner

chef habitat is also an open-source automation framework which can help you to define, packaging and delivering an application to any environment of your choice regardless of the operating system.

Terraform uses the `habitat provisioner` and it also uses the `ssh` for file transfer.

Here is an example code snippet for the same -

```BASH
resource "aws_instance" "habitat_provisioner" {
  count = 1

  provisioner "habitat" {
    peers = [aws_instance.habitat_provisioner[0].private_ip]
    use_sudo = true
    service_type = "systemd"
    accept_license = true

    service {
      name = "core/redis"
      topology = "leader"
      user_toml = file("conf/redis.toml")
    }
  }
}
```

Note: Reference taken from terraform.io

6. puppet provisioner

With the help of puppet provisioner, you can install and configure the puppet agent on the remote machine and it also supports the `ssh` and `winrm`

But few things which you need to consider before using the puppet provisioners -

1. You should make sure that `ssh` and `cURL` is available on the remote host
2. In case you are using the `winrm` then you should make sure that `PowerShell 2.0` is available on the remote host.
3. Third you need to install the Bolt onto your workstation or local machine before you start using the puppet provisioner

Here is an example of a puppet provisioner -

```BASH
resource "aws_instance" "web" {
  provisioner "puppet" {
    server             = aws_instance.puppetmaster.public_dns
    server_user        = "ubuntu"
    extension_requests = {
      pp_role = "webserver"
    }
  }
}
```

7. salt-masterless

salt-masterless provisions the machine build by Terraform suing Salt states. I have not used the *salt-masterless* provisioners, so could not add more details here.

But if you are interested then here is an example code snippet that I found from terraform.io

```BASH

provisioner "salt-masterless" {
    "local_state_tree" = "/srv/salt"
}
```

A company is planning to securely manage one-time fixed license keys in AWS. The company's development team needs to access the license keys in automaton scripts that run in Amazon EC2 instances and in AWS CloudFormation stacks.
Which solution will meet these requirements MOST cost-effectively?

•A. Amazon S3 with encrypted files prefixed with “config”

•B. AWS Secrets Manager secrets with a tag that is named SecretString

•C. AWS Systems Manager Parameter Store SecureString parameters

•D. CloudFormation NoEcho parameters

An application runs on an Amazon EC2 instance in a VPC. The application processes logs that are stored in an Amazon S3 bucket. The EC2 instance needs to access the S3 bucket without connectivity to the internet.
Which solution will provide private network connectivity to Amazon S3?

•A. Create a gateway VPC endpoint to the S3 bucket.

•B. Stream the logs to Amazon CloudWatch Logs. Export the logs to the S3 bucket.

•C. Create an instance profile on Amazon EC2 to allow S3 access.

•D. Create an Amazon API Gateway API with a private link to access the S3 endpoint.

A company is hosting a web application on AWS using a single Amazon EC2 instance that stores user-uploaded documents in an Amazon EBS volume. For better scalability and availability, the company duplicated the architecture and created a second EC2 instance and EBS volume in another Availability Zone, placing both behind an Application Load Balancer. After completing this change, users reported that, each time they refreshed the website, they could see one subset of their documents or the other, but never all of the documents at the same time.
What should a solutions architect propose to ensure users see all of their documents at once?

•A. Copy the data so both EBS volumes contain all the documents

•B. Configure the Application Load Balancer to direct a user to the server with the documents

•C. Copy the data from both EBS volumes to Amazon EFS. Modify the application to save new documents to Amazon EFS

•D. Configure the Application Load Balancer to send the request to both servers. Return each document from the correct server

A developer creates a VPC named VPC-A that has public and private subnets. The developer also creates an Amazon RDS database inside the private subnet of VPC-A. To perform some queries, the developer creates an AWS Lambda function in the default VPC. The Lambda function has code to access the RDS database. When the Lambda function runs, an error message indicates that the function cannot connect to the RDS database.

How can the developer solve this problem?

•A. Modify the RDS security group. Add a rule to allow traffic from all the ports from the VPC CIDR block.

•B. Redeploy the Lambda function in the same subnet as the RDS instance. Ensure that the RDS security group allows traffic from the Lambda function.

•C. Create a security group for the Lambda function. Add a new rule in the RDS security group to allow traffic from the new Lambda security group.

•D. Create an IAM role. Attach a policy that allows access to the RDS database. Attach the role to the Lambda function.

A company needs to ingest terabytes of data each hour from thousands of sources that are delivered almost continually throughout the day. The volume of messages generated varies over the course of the day. Messages must be delivered in real time for fraud detection and live operational dashboards.
Which approach will meet these requirements?

•A. Send the messages to an Amazon SQS queue, then process the messages by using a fleet of Amazon EC2 instances

•B. Use the Amazon S3 API to write messages to an S3 bucket, then process the messages by using Amazon Redshift

•C. Use AWS Data Pipeline to automate the movement and transformation of data

•D. Use Amazon Kinesis Data Streams with Kinesis Client Library to ingest and deliver messages

A developer is using Amazon S3 as the event source that invokes a Lambda function when new objects are created in the bucket. The event source mapping information is stored in the bucket notification configuration. The developer is working with different versions of the Lambda function, and has a constant need to update notification configuration so that Amazon S3 invokes the correct version.
What is the MOST efficient and effective way to achieve mapping between the S3 event and Lambda?

- A. Use a different Lambda trigger.
- B. Use Lambda environment variables.
- C. Use a Lambda alias.
- D. Use Lambda tags.

A developer is creating an AWS Lambda function in VPC mode. An Amazon S3 event will invoke the Lambda function when an object is uploaded into an S3 bucket. The Lambda function will process the object and produce some analytic results that will be recorded into a file. Each processed object will also generate a log entry that will be recorded into a file.

Other Lambda functions, AWS services, and on-premises resources must have access to the result files and log file. Each log entry must also be appended to the same shared log file. The developer needs a solution that can share files and append results into an existing file.

Which solution should the developer use to meet these requirements?

•A. Create an Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system in Lambda. Store the result files and log file in the mount point. Append the log entries to the log file.

•B. Create an Amazon Elastic Block Store (Amazon EBS) Multi-Attach enabled volume. Attach the EBS volume to all Lambda functions. Update the Lambda function code to download the log file, append the log entries, and upload the modified log file to Amazon EBS.

•C. Create a reference to the /tmp local directory. Store the result files and log file by using the directory reference. Append the log entry to the log file.

•D. Create a reference to the /opt storage directory. Store the result files and log file by using the directory reference. Append the log entry to the log file.

A company needs to set up secure database credentials for all its AWS Cloud resources. The company's resources include Amazon RDS DB instances, Amazon DocumentDB clusters, and Amazon Aurora DB instances. The company's security policy mandates that database credentials be encrypted at rest and rotated at a regular interval.

Which solution will meet these requirements MOST securely?

•A. Set up IAM database authentication for token-based access. Generate user tokens to provide centralized access to RDS DB instances, Amazon DocumentDB clusters, and Aurora DB instances.

•B. Create parameters for the database credentials in AWS Systems Manager Parameter Store. Set the Type parameter to SecureString. Set up automatic rotation on the parameters.

•C. Store the database access credentials as an encrypted Amazon S3 object in an S3 bucket. Block all public access on the S3 bucket. Use S3 server-side encryption to set up automatic rotation on the encryption key.

•D. Create an AWS Lambda function by using the SecretsManagerRotationTemplate template in the AWS Secrets Manager console. Create secrets for the database credentials in Secrets Manager. Set up secrets rotation on a schedule.